

ROUNSAVILLE TECHNOLOGIES

R-COREX INTELLIGENCE ARCHITECTURE

MASTER INTELLECTUAL PROPERTY, TECHNICAL PROVENANCE, AND ARCHITECTURE DOSSIER

Document ID:	RT-IP-RCOREX-MASTER-001
Version:	1.1.0
Status:	Master Architecture & IP Provenance Record
Original Author / Principal Architect:	Joseph Michael Rounsaville
Organization:	Rounsaville Technologies
Initial Record Date:	July 22, 2026
This Revision Date:	July 23, 2026
Document Classification:	Proprietary / Confidential — Internal Master Record
Architecture Family:	R-CoreX Intelligence Architecture
Companion Record:	RT-IP-RCOREX-MAPPING-002 v1.0.0

R-COREX INTELLIGENCE ARCHITECTURE

MASTER INTELLECTUAL PROPERTY, TECHNICAL PROVENANCE, AND ARCHITECTURE DOSSIER

Document ID: RT-IP-RCOREX-MASTER-001 **Version:** 1.1.0 **Status:** Master Architecture & IP Provenance Record **Original Author / Principal Architect:** Joseph Michael Rounsaville **Organization:** Rounsaville Technologies **Initial Record Date:** July 22, 2026 **This Revision Date:** July 23, 2026
Document Classification: Proprietary / Confidential — Internal Master Record **Architecture Family:** R-CoreX Intelligence Architecture

VERSION HISTORY

Version	Date	Summary
1.0.0	2026-07-22	Initial master dossier. R-CoreX described as a conceptual, aspirational architecture; Section 16 code-to-concept mapping was a placeholder — paths were not yet mapped to any real repository.
1.1.0	2026-07-23	Reconceptualization. Section 16 replaced with a real, evidence-based mapping against the actual Rounsaville Technologies GitHub repositories (see Section 13 additions and the companion record RT - IP - RCOREX - MAPPI NG - 002 _ v1 . 0 . 0). Components once described only in the abstract are now reclassified by their true implementation status — Implemented, Scaffolded/Planned, or No Corresponding Code — instead of being presented uniformly as "proposed."

DOCUMENT CONTROL

1. Purpose

This dossier establishes a centralized, version-controlled record of the conceptual architecture, technical organization, authorship, provenance, ownership claims, extensibility model, and intellectual-property

structure associated with the R-CoreX Intelligence Architecture and its associated systems.

The purpose of this document is to create a durable reference describing:

- What each architectural component is.
- Who created or authored the architecture and associated materials.
- When the architecture and components were documented.
- How the components interact.
- Which elements are original Rounsaville Technologies work versus third-party dependencies.
- Which software, repositories, modules, and codebases implement each concept.
- Which names and marks are intended to function as trademarks or product identifiers.
- Who owns or claims ownership of the intellectual property.
- Which external libraries, frameworks, APIs, and dependencies are governed by third-party licenses.
- How future contributors are expected to assign or license intellectual-property rights.
- How the system is versioned, extended, audited, and maintained.

This dossier is intended to function as a technical provenance and intellectual-property management record.

It is not, by itself, a substitute for legal advice, a patent application, a trademark registration, a copyright registration, or a signed intellectual-property assignment agreement.

2. OWNERSHIP DECLARATION

The architecture described in this document is associated with:

Rounsaville Technologies

and is attributed to:

Joseph Michael Rounsaville

as the stated founder, principal architect, and originating author of the R-CoreX architectural framework and associated conceptual systems described herein.

Unless otherwise documented through a written agreement, third-party license, employment agreement, contractor agreement, or other legally controlling instrument, original materials created by Joseph Michael Rounsaville for Rounsaville Technologies are intended to be maintained as intellectual property of Rounsaville Technologies or its lawful successor or assignee.

This record establishes an authorship and provenance claim based on the documentation date and version history contained herein.

Ownership of individual assets may vary depending upon:

- the legal entity that owns the asset;
- the circumstances under which the asset was created;
- employment or contractor agreements;
- contributor agreements;
- open-source licenses;
- third-party software licenses;
- patent assignments;
- trademark registrations;
- copyright registrations;

- applicable law.

3. MASTER ARCHITECTURE TREE

```

ROUNSAVILLE TECHNOLOGIES
|
v
R-COREX INTELLIGENCE ARCHITECTURE
|
+-- Nine-Phase Interpretation Pipeline
|
+-- Verified Bus
|
+-- IHI
|   +-- Extensible Intelligence Layer
|
+-- RHI
|   +-- Extensible Reasoning Layer
|
+-- NHI
|   +-- Extensible Natural/Human Interface Layer
|
+-- SRI
|   +-- Extensible Sensory/Resonance Layer
|
+-- Telemetry & Event Fabric
|
+-- Event Schemas
+-- Identity
+-- Provenance
+-- Verification
+-- Audit Trails
+-- Extension APIs

```

The architecture is organized as a hierarchical intelligence and telemetry framework.

At the highest level, Rounsaville Technologies represents the organizational ownership and product ecosystem.

The R-CoreX Intelligence Architecture represents the parent technical architecture.

The architecture is then divided into processing, verification, intelligence, reasoning, interface, sensory, and telemetry domains.

4. R-COREX INTELLIGENCE ARCHITECTURE

4.1 Definition

R-CoreX is the designated name for the central architectural framework that coordinates the processing, interpretation, verification, reasoning, interface, sensory, and telemetry functions of the Rounsaville Technologies ecosystem.

R-CoreX is intended to function as an extensible architecture rather than a single monolithic application.

Its purpose is to provide a common structural foundation through which multiple intelligence systems and software products can exchange:

- observations;
- events;
- context;

- interpretations;
- reasoning states;
- identity information;
- verification results;
- telemetry;
- provenance;
- audit records.

4.2 Architectural Role

R-CoreX serves as the parent architecture for:

- Nine-Phase Interpretation Pipeline;
- Verified Bus;
- IHI;
- RHI;
- NHI;
- SRI;
- Telemetry & Event Fabric.

4.3 Intended Technical Characteristics

R-CoreX is designed around:

- modularity;
- extensibility;
- event-driven communication;
- provenance;
- verification;
- auditability;
- identity-aware processing;
- component isolation;
- version control;
- API-based integration;
- replaceable modules.

5. NINE-PHASE INTERPRETATION PIPELINE

5.1 Definition

The Nine-Phase Interpretation Pipeline is the primary cognitive and interpretive processing sequence within the R-CoreX architecture.

The nine phases are:

- 1 Observation
- 2 Interpretation
- 3 Alignment
- 4 Reasoning
- 5 Knowledge Retrieval
- 6 Relationship Mapping
- 7 Inference

- 8 Response Construction
- 9 Evolution

5.2 Phase Descriptions

Phase 1 — Observation

The system identifies and receives available signals, inputs, events, or environmental information.

Potential inputs include:

- text;
- telemetry;
- sensor data;
- audio;
- visual information;
- system events;
- user interactions;
- external APIs.

Phase 2 — Interpretation

Raw observations are transformed into structured meaning.

This phase may include:

- classification;
- extraction;
- normalization;
- semantic interpretation;
- contextualization.

Phase 3 — Alignment

The interpreted information is compared against:

- system objectives;
- user intent;
- current state;
- known constraints;
- safety requirements;
- operational goals.

Phase 4 — Reasoning

The system evaluates relationships between known information and determines possible conclusions, actions, or next steps.

Phase 5 — Knowledge Retrieval

Relevant internal or external knowledge is retrieved and incorporated into the reasoning process.

Phase 6 — Relationship Mapping

Entities, events, concepts, and dependencies are connected into an explicit relationship model.

Phase 7 — Inference

The system derives conclusions from available evidence, while distinguishing:

- observed facts;
- reasonable conclusions;
- possibilities;
- unknowns.

Phase 8 — Response Construction

The system converts the resulting state into an appropriate output.

Outputs may include:

- text;
- commands;
- telemetry;
- visualizations;
- API responses;
- system actions.

Phase 9 — Evolution

The system evaluates outcomes and updates future processing based on:

- feedback;
- new knowledge;
- system performance;
- detected errors;
- changing context.

6. VERIFIED BUS

6.1 Definition

The Verified Bus is the proposed event and communication layer responsible for moving information between R-CoreX components while maintaining a structured record of event identity, provenance, verification state, and audit history.

6.2 Core Functions

The Verified Bus is intended to provide:

- event transport;
- message identification;
- source attribution;
- timestamping;
- provenance tracking;
- validation;
- integrity verification;
- audit logging;
- routing;
- subscription mechanisms.

6.3 Conceptual Event Model

EVENT

```
|
+-- Event ID
+-- Source
+-- Timestamp
+-- Event Type
+-- Payload
+-- Context
+-- Provenance
+-- Verification State
+-- Integrity Metadata
+-- Audit Record
```

6.4 Verification Principle

An event should be capable of being evaluated according to:

- WHO produced it?
- WHAT was produced?
- WHEN was it produced?
- WHERE did it originate?
- WHY was it generated?
- HOW was it transformed?
- WHAT verified it?
- WHAT happened afterward?

This creates a foundation for traceable system behavior.

7. IHI — EXTENSIBLE INTELLIGENCE LAYER

7.1 Definition

IHI is designated as the Extensible Intelligence Layer.

Its purpose is to provide modular intelligence capabilities that can be attached to the R-CoreX architecture.

7.2 Potential Responsibilities

IHI may include:

- intelligence modules;
- classification;
- pattern recognition;
- context analysis;
- decision support;
- model orchestration;
- intelligence scoring;
- adaptive processing.

7.3 Extensibility

IHI modules should be independently replaceable and versioned.

Conceptually:


```
IHI CORE
|
+-- Intelligence Module A
+-- Intelligence Module B
+-- Intelligence Module C
+-- Future Extensions
```

8. RHI — EXTENSIBLE REASONING LAYER

8.1 Definition

RHI is designated as the Extensible Reasoning Layer.

RHI provides structured reasoning capabilities within R-CoreX.

8.2 Potential Responsibilities

RHI may manage:

- reasoning chains;
- hypothesis generation;
- evidence evaluation;
- dependency analysis;
- conflict detection;
- decision logic;
- uncertainty representation.

8.3 Evidence Model

The RHI framework should distinguish:

```
OBSERVED FACT
|
v
SUPPORTED CONCLUSION
|
v
POSSIBLE EXPLANATION
|
v
UNKNOWN / UNVERIFIED
```

This distinction is intended to reduce unsupported assumptions.

9. NHI — EXTENSIBLE NATURAL/HUMAN INTERFACE LAYER

9.1 Definition

NHI is designated as the Extensible Natural/Human Interface Layer.

NHI provides the interface between humans and the R-CoreX architecture.

9.2 Potential Interfaces

NHI may support:

- natural language;

- voice;
- visual interfaces;
- dashboards;
- conversational systems;
- command interfaces;
- accessibility systems;
- multimodal interaction.

9.3 Human Context

NHI is intended to preserve:

- user intent;
- conversational context;
- interaction history;
- interface state;
- permissions;
- identity.

10. SRI — EXTENSIBLE SENSORY/RESONANCE LAYER

10.1 Definition

SRI is designated as the Extensible Sensory/Resonance Layer.

SRI provides the architectural boundary for systems that ingest and interpret sensory, signal, environmental, or resonance-related information.

10.2 Potential Inputs

SRI may process:

- audio;
- visual signals;
- spatial data;
- telemetry;
- environmental sensors;
- frequency data;
- other machine-readable signals.

10.3 Extensibility

SRI is intended to support modular sensor and signal-processing extensions.

```
SRI
|
+-- Audio
+-- Visual
+-- Spatial
+-- Environmental
+-- Frequency
+-- Future Sensors
```

11. TELEMETRY & EVENT FABRIC

The Telemetry & Event Fabric provides the system-wide observational and historical layer.

It is composed of:

```
Telemetry & Event Fabric
|
+-- Event Schemas
+-- Identity
+-- Provenance
+-- Verification
+-- Audit Trails
+-- Extension APIs
```

11.1 Event Schemas

Event schemas define the structure of system messages.

Each event may contain:

- event identifier;
- event type;
- source;
- destination;
- timestamp;
- payload;
- context;
- version;
- provenance;
- verification status.

11.2 Identity

Identity establishes the origin of:

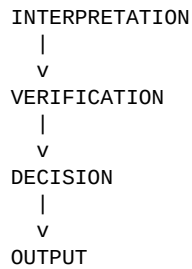
- users;
- services;
- devices;
- modules;
- agents;
- system components.

11.3 Provenance

Provenance records the history of information.

A provenance chain may describe:

```
SOURCE
|
v
INGESTION
|
v
TRANSFORMATION
|
v
```



11.4 Verification

Verification records whether an event, claim, or data element has been:

- validated;
- independently confirmed;
- cryptographically authenticated;
- source-attributed;
- unverified.

11.5 Audit Trails

Audit trails provide historical records of system operations.

Audit information may include:

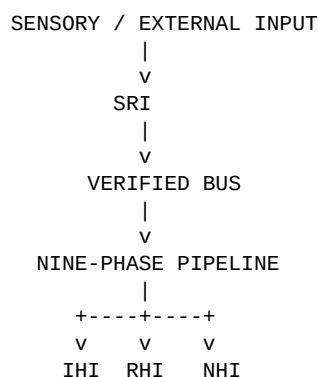
- actor;
- action;
- timestamp;
- affected object;
- previous state;
- resulting state;
- authorization;
- verification status.

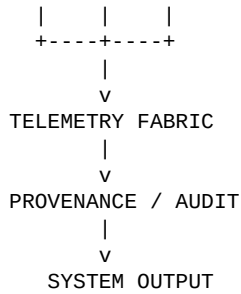
11.6 Extension APIs

Extension APIs allow new modules to integrate without rewriting the entire architecture.

12. COMPONENT INTERACTION MODEL

The primary interaction model is:





The architecture is intended to be bidirectional.

Outputs may generate new events that return to the system as new observations, allowing the system to evolve through continuous feedback.

13. IP ASSET REGISTRY

ID	Asset	Category	Status (v1.0.0)	Status (v1.1.0 — evidence-based)
RT-IP-0001	R-CoreX Core Architecture	Architecture	Proprietary	Proprietary — conceptual layer; concretely hosted by RealityStateOS's 14-module system

ID	Asset	Category	Status (v1.0.0)	Status (v1.1.0 — evidence-based)
RT-IP-0002	Nine-Phase Interpretation Pipeline	Method / Architecture	Proprietary	Proprietary — Partially Implemented. Real code implements a 5-step cognition loop (Observe → Interpret → Align → Reason → Evolve); three of the nine dossier phases (Knowledge Retrieval, Relationship Mapping, Inference) are folded into the single "Reason" step rather than kept distinct, and Response Construction is not a discrete loop step
RT-IP-0003	Verified Bus	Software Architecture	Proprietary	Proprietary — Prototype Implemented / Full Module Planned. Prototype: <code>event_bus.py</code> (synchronous pub/sub). Dedicated 004_VERIFIED_BUS module (broker, consensus, verification, telemetry) not yet built

ID	Asset	Category	Status (v1.0.0)	Status (v1.1.0 — evidence-based)
RT-IP-0004	IHI	Architecture	Proprietary	Proprietary — No dedicated module . Intelligence/pattern-recognition duties are distributed across knowledge_graph.py, semantic_index.py, vector_store.py; not separated from RHI in real code
RT-IP-0005	RHI	Architecture	Proprietary	Proprietary — Implemented (prototype) . reasoning.py + inference.py (forward-chaining rule engine over Findings)
RT-IP-0006	NHI	Architecture	Proprietary	Proprietary — Prototype Implemented / Full Module Planned . Prototype: api.py, rest.py, grpc.py, websocket.py, cli.py, dashboard.py. Full surface is planned module 011_INTERFACE

ID	Asset	Category	Status (v1.0.0)	Status (v1.1.0 — evidence-based)
RT-IP-0007	SRI	Architecture	Proprietary	Proprietary — Minimal Implementation / Module Planned. observation.py performs raw, uninterpreted input intake only; dedicated sensory/signal module 008_REALITY_TAPE not yet built
RT-IP-0008	Telemetry & Event Schema	Data Architecture	Proprietary	Proprietary — Implemented. 003_FOUNDATION/CoreTypes/src/event.ts (hash-chained, signed event envelope); metrics.py, monitoring.py
RT-IP-0009	Extensibility Framework	Software Architecture	Proprietary	Proprietary — Implemented (prototype). plugins.py plugin/extension registry
RT-IP-0010	Provenance Model	Data Architecture	Proprietary	Proprietary — Implemented. provenance.py, version_control.py

ID	Asset	Category	Status (v1.0.0)	Status (v1.1.0 — evidence-based)
RT-IP-0011	Verification Model	Technical Architecture	Proprietary	Proprietary — Implemented. 003_FOUNDATION/CoreTypes/src/crypto.ts + event.ts (verifyEvent, verifyChain); security.py, authentication.py, authorization.py, encryption.py
RT-IP-0012	Audit Trail Model	Technical Architecture	Proprietary	Proprietary — Implemented. version_control.py (git-lite, append-only history)
RT-IP-0013	Extension API Model	Software Architecture	Proprietary	Proprietary — Implemented (prototype). plugins.py
RT-IP-0014	RealityStateOS — Host Product Implementation	Product / Codebase	*(new in v1.1.0)*	Proprietary — 14-module architecture (presencefieldowner-glitch/RealityStateOS); Architecture, Kernel, and Foundation modules built and tested (33/33 TS tests passing); remaining 11 modules scaffolded in the module map but not yet built, per the repository's own documented build order

ID	Asset	Category	Status (v1.0.0)	Status (v1.1.0 — evidence-based)
RT-IP-0015	LakeTiticaca — Kernel AI Cognition Loop	Software Architecture	*(new in v1.1.0)*	Proprietary — RealityStateOS/002_KERNEL/LakeTiticaca/laketiticaca/; single-node, in-memory prototype implementing observation, interpretation, alignment, reasoning, retrieval, relationship, inference, evolution, provenance, and version-control modules

The above registry is a working internal classification system and should be expanded as individual repositories, source files, algorithms, specifications, and products are identified. Full supporting evidence (file paths, commit references) for every "Implemented" status above is recorded in the companion mapping record RT-IP-RCOREX-MAPPING-002_v1.0.0.

14. AUTHORSHIP RECORD

- **Primary Author:** Joseph Michael Rounsaville
- **Organization:** Rounsaville Technologies
- **Role:** Founder / Principal Architect / Originating Author
- **Initial Documentation Date:** July 22, 2026
- **This Revision:** Version 1.1.0, July 23, 2026

This document records the stated authorship and architectural provenance of the systems described herein.

Future contributors must be separately documented.

15. ORIGINAL VS. THIRD-PARTY TECHNOLOGY

15.1 Original Technology

The following categories are intended to represent original Rounsaville Technologies work where independently created:

- architectural specifications;
- original source code;
- original algorithms;
- original data schemas;
- original system designs;
- original documentation;
- original UI designs;
- original branding;
- original proprietary interfaces;
- original telemetry models.

15.2 Third-Party Technology

Third-party components may include:

- operating systems;
- programming languages;
- web browsers;
- frameworks;
- libraries;
- databases;
- cloud services;
- APIs;
- AI models;
- fonts;
- development tools;
- open-source software.

Each third-party dependency must retain its applicable license and attribution.

The use of third-party software does not automatically transfer ownership of that software to Rounsaville Technologies.

16. CODE-TO-CONCEPT MAPPING (REVISED — REAL REPOSITORIES)

This section is superseded from the v1.0.0 placeholder. As of this revision, every path below is a real, git-verified location in a Rounsaville Technologies repository — not an architectural placeholder. Full evidence (docstrings, commit references, exact file contents) is maintained in the companion record RT-IP-RCOREX-MAPPING-002_v1.0.0; this section carries the summary.

16.1 Repositories surveyed

Repository	Default Branch / Commit	Real-World Role
presencefieldowner-glitch/RealityStateOS	main @ 8e41ae2	Primary host of R-CoreX. A 14-module OS architecture (000–999) that independently arrived at nearly the same shape as this dossier — including a module literally named 004_VERIFIED_BUS.
presencefieldowner-glitch/rounsaville-ai-music-generator	—	Empty. Zero commits exist. No R-CoreX component has any implementation here. Flagged for re-mapping once code is pushed.
presencefieldowner-glitch/presence-resonance	main @ a3f9727	Sibling product, not an R-CoreX host. A genealogy/family-history platform by the same author. Every package is an unimplemented export <code>{}</code> ; stub; naming overlap with "resonance" (SRI) and "ai-interpreter" (IHI) is nominal only, not functional.

16.2 Reconceptualized definition

R-CoreX is not a separate, freestanding codebase. It is the **abstract description of RealityStateOS's own architecture**, authored and later restated in general terms. RealityStateOS's module map (README.md) already names its event backbone "Verified Bus" (module 004) using almost the same language as Section 6 of this dossier, and its Kernel-level AI subsystem — **LakeTiticaca** — independently implements a condensed cognition loop that is the working ancestor of the Nine-Phase Interpretation Pipeline:

```
Dossier (v1.0.0, aspirational): Observation -> Interpretation -> Alignment -> Reasoning
                                -> Knowledge Retrieval -> Relationship Mapping -> Inference
                                -> Response Construction -> Evolution    (9 phases)

LakeTiticaca (real, tested code): Observe -> Interpret -> Align -> Reason -> Evolve    (5 steps)
    ^ Knowledge Retrieval, Relationship Mapping, and Inference
    ^ are implemented as building blocks *inside* "Reason",
    ^ not as separate loop phases.
    ^ Response Construction is not a cognition-loop step at all –
    ^ it is handled by separate interface-surface modules
    (api.py / rest.py / grpc.py / websocket.py / cli.py / dashboard.py).
```

16.3 Concept-to-code mapping table

R-CoreX Concept	Real Path(s)	Status
R-CoreX (parent architecture)	RealityStateOS/ (whole repo, 14-module map in README.md)	Host product — scaffolding phase
Nine-Phase Pipeline — Observation	002_KERNEL/LakeTiticaca /laketiticaca/observati on.py	Implemented (prototype)
Nine-Phase Pipeline — Interpretation	002_KERNEL/LakeTiticaca /laketiticaca/interpret ation.py	Implemented (prototype)
Nine-Phase Pipeline — Alignment	002_KERNEL/LakeTiticaca /laketiticaca/alignment .py	Implemented (prototype) — governance/policy gate
Nine-Phase Pipeline — Reasoning	002_KERNEL/LakeTiticaca /laketiticaca/reasoning .py	Implemented (prototype)
Nine-Phase Pipeline — Knowledge Retrieval	002_KERNEL/LakeTiticaca /laketiticaca/retrieval .py (unifies semantic_index.py, vector_store.py, knowledge_graph.py)	Implemented (prototype) — collapsed into "Reason" step, not a separate loop phase
Nine-Phase Pipeline — Relationship Mapping	002_KERNEL/LakeTiticaca /laketiticaca/relations hips.py; 003_FOUNDATION/ EntityModel/src/entityM odel.ts	Implemented (prototype) — same caveat
Nine-Phase Pipeline — Inference	002_KERNEL/LakeTiticaca /laketiticaca/inference .py (forward-chaining rule engine)	Implemented (prototype) — same caveat
Nine-Phase Pipeline — Response Construction	002_KERNEL/LakeTiticaca /laketiticaca/api.py, rest.py, grpc.py, websocket.py, cli.py, dashboard.py	Implemented (prototype) — not modeled as a discrete cognition-loop step
Nine-Phase Pipeline — Evolution	002_KERNEL/LakeTiticaca /laketiticaca/evolution .py (layered on version_control.py)	Implemented (prototype)

R-CoreX Concept	Real Path(s)	Status
Verified Bus	Prototype: 002_KERNEL/LakeTiticaca/laketiticaca/event_bus.py (synchronous pub/sub). Full module: 004_VERIFIED_BUS/ (not yet scaffolded)	Prototype implemented; full module planned
IHI (Extensible Intelligence Layer)	No dedicated module — distributed across knowledge_graph.py, semantic_index.py, vector_store.py	No dedicated implementation
RHI (Extensible Reasoning Layer)	reasoning.py + inference.py	Implemented (prototype)
NHI (Extensible Natural/Human Interface Layer)	Prototype: api.py, rest.py, grpc.py, websocket.py, cli.py, dashboard.py. Full surface: 011_INTERFACE/ (not yet scaffolded)	Prototype implemented; full module planned
SRI (Extensible Sensory/Resonance Layer)	Minimal: observation.py (raw input intake only). Full module: 008_REALITY_TAPE/ (sensor ingestion, timeline replay — not yet scaffolded)	Minimal implementation; full module planned
Telemetry & Event Fabric — Event Schemas	003_FOUNDATION/CoreTypes/src/event.ts (hash-chained, signed Event<T> envelope, RFC-0002 §5)	Implemented
Telemetry & Event Fabric — Identity	002_KERNEL/LakeTiticaca/laketiticaca/authentication.py, authorization.py (HMAC-signed identity tokens). Full module: 007_PRESENCE/ (not yet scaffolded)	Prototype implemented; full module planned
Telemetry & Event Fabric — Provenance	002_KERNEL/LakeTiticaca/laketiticaca/provenance.py	Implemented

R-CoreX Concept	Real Path(s)	Status
Telemetry & Event Fabric — Verification	003_FOUNDATION/CoreTypes/src/crypto.ts + event.ts (verifyEvent, verifyChain); security.py, encryption.py	Implemented
Telemetry & Event Fabric — Audit Trails	002_KERNEL/LakeTiticaca/laketiticaca/version_control.py (git-lite, append-only)	Implemented
Telemetry & Event Fabric — Extension APIs	002_KERNEL/LakeTiticaca/laketiticaca/plugins.py	Implemented (prototype)

16.4 Honest gaps

- **IHI has no dedicated real module.** The dossier's split between an "Intelligence" layer (IHI) and a "Reasoning" layer (RHI) does not exist in the real code — LakeTiticaca treats retrieval/pattern-matching as inputs to reasoning, not as a separate subsystem. Any future IP or trademark claim over IHI specifically should be understood as aspirational, not yet reduced to practice.
- rounsaville-ai-music-generator **implements none of this architecture** — it is an empty repository. Nothing in RT-IP-0001 through RT-IP-0013 can currently be evidenced there.
- presence-resonance **is not an R-CoreX asset.** It is a related but separate product by the same author; its "resonance" and "ai-interpreter" package names should not be cited as evidence of SRI or IHI implementation — both are empty stubs.
- All LakeTiticaca and Foundation modules cited above are explicitly documented in-repo as **single-node, in-memory prototypes**, not production/distributed implementations. replication.py and synchronization.py "define the seams" for future distributed behavior but do not implement it.

17. VERSION CONTROL REQUIREMENTS

All proprietary code and architecture documentation should be maintained under version control.

Recommended metadata:

- Project:
- Repository:
- Commit:
- Version:
- Author:
- Date:
- Change Description:
- Related IP Asset:
- Related Specification:
- Dependencies:
- License:

Each significant change should produce a new version record.

Recommended semantic versioning: MAJOR.MINOR.PATCH

Example:

- R-CoreX 1.0.0
- R-CoreX 1.1.0
- R-CoreX 1.1.1
- R-CoreX 2.0.0

18. TRADEMARK AND BRAND REGISTRY

The following names are designated as potential product, technology, or architectural marks.

Mark	Intended Use	Status
Rounsaville Technologies	Company / Brand	Ownership to be verified
R-CoreX	Core Architecture	Proposed / Claimed
Nine-Phase Interpretation Pipeline	Method / Architecture	Descriptive designation
Verified Bus	Technical Component	Proposed
IHI	Intelligence Layer	Proposed
RHI	Reasoning Layer	Proposed
NHI	Natural/Human Interface	Proposed
SRI	Sensory/Resonance Layer	Proposed

Trademark availability should be independently searched before commercial use or registration.

Trademark rights are jurisdiction-specific and should not be assumed solely from inclusion in this document.

19. THIRD-PARTY LICENSE REGISTER

A live dependency register should be maintained.

- Dependency:
- Version:
- Purpose:
- Repository:
- License:
- Copyright Holder:
- Required Attribution:
- Modification:
- Distribution Restrictions:
- Commercial Use:
- Notes:

Potential categories include:

- MIT;
- Apache 2.0;
- BSD;
- GPL;
- LGPL;
- MPL;
- proprietary commercial licenses;
- cloud service agreements;
- API terms of service.

No third-party dependency should be classified as proprietary Rounsaville Technologies IP merely because it is integrated into R-CoreX.

20. CONTRIBUTOR INTELLECTUAL PROPERTY POLICY

Any contributor to Rounsaville Technologies software or architecture should be subject to a written agreement.

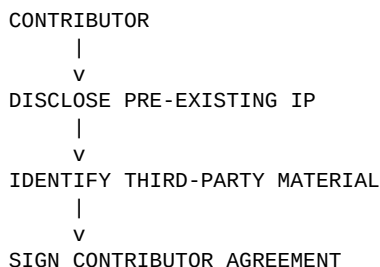
The agreement should address:

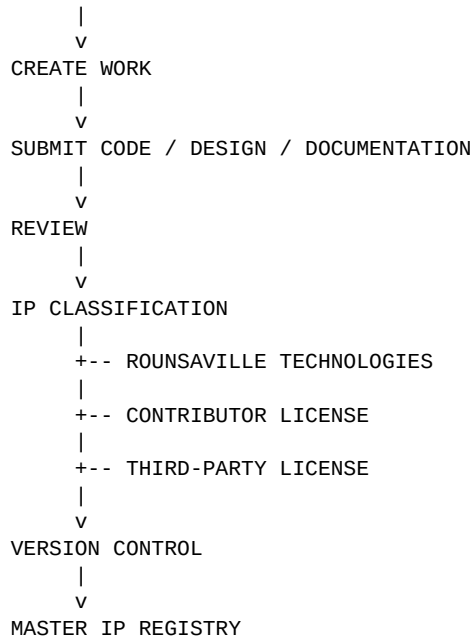
- 1 Ownership of newly created work.
- 2 Assignment of intellectual-property rights.
- 3 Confidentiality.
- 4 Trade secrets.
- 5 Pre-existing intellectual property.
- 6 Open-source contributions.
- 7 Third-party materials.
- 8 Moral rights where applicable.
- 9 Patent rights.
- 10 Copyright.
- 11 Trademark use.
- 12 Licensing.
- 13 Termination.
- 14 Survival of obligations.

A contributor should not contribute third-party proprietary material without authorization.

A contributor should disclose any pre-existing intellectual property incorporated into the project.

21. CONTRIBUTOR IP FLOW

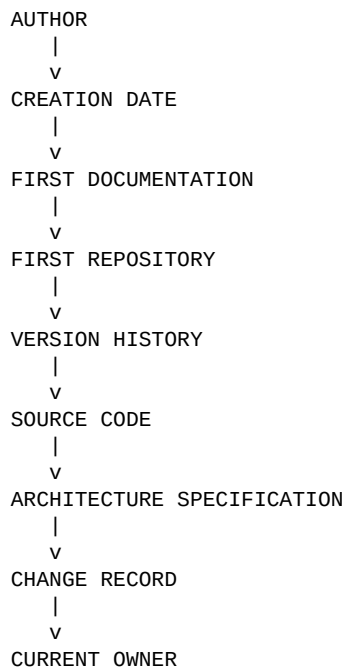




The exact legal mechanism — assignment, exclusive license, non-exclusive license, or other structure — should be determined through appropriate legal agreements.

22. PROVENANCE STANDARD

Every significant intellectual-property asset should be traceable through:



The objective is to establish a continuous chain of provenance.

23. MASTER IP EVIDENCE PACKAGE

The following records should be preserved:

- architecture specifications;
- source code repositories;
- Git commit histories;
- signed release tags;
- design files;
- technical diagrams;
- dated PDFs;
- project notebooks;
- invention disclosures;
- contributor agreements;
- contractor agreements;
- employment agreements;
- copyright registrations;
- trademark filings;
- patent filings;
- third-party licenses;
- dependency manifests;
- software bills of materials;
- release archives.

Recommended preservation formats include:

- PDF/A
- Git Repository
- Signed Git Tags
- SHA-256 Hashes
- Encrypted Backup
- Offline Archive

24. RECOMMENDED MASTER REPOSITORY STRUCTURE

```
Rounsaville-Technologies/
|
+-- 00-IP-REGISTRY/
|   +-- master-ip-registry.md
|   +-- asset-index.csv
|   +-- ownership-records/
|
+-- 01-R-COREX/
|   +-- architecture/
|   +-- specifications/
|   +-- diagrams/
|
+-- 02-NINE-PHASE-PIPELINE/
|   +-- observation/
|   +-- interpretation/
|   +-- alignment/
|   +-- reasoning/
|   +-- knowledge/
|   +-- relationships/
|   +-- inference/
|   +-- response/
|   +-- evolution/
```

```

|
+-- 03-VERIFIED-BUS/
|
+-- 04-IHI/
|
+-- 05-RHI/
|
+-- 06-NHI/
|
+-- 07-SRI/
|
+-- 08-TELEMETRY/
|   +-- schemas/
|   +-- identity/
|   +-- provenance/
|   +-- verification/
|   +-- audit/
|
+-- 09-APIS/
|
+-- 10-THIRD-PARTY-LICENSES/
|
+-- 11-CONTRIBUTORS/
|
+-- 12-RELEASES/
|
+-- 13-LEGAL/

```

25. CHANGE CONTROL

Any modification to the R-CoreX architecture should be recorded.

Each change should include:

- CHANGE ID:
- DATE:
- AUTHOR:
- VERSION:
- COMPONENT:
- DESCRIPTION:
- REASON:
- AFFECTED SYSTEMS:
- DEPENDENCIES:
- IP IMPACT:
- SECURITY IMPACT:
- BACKWARD COMPATIBILITY:
- APPROVAL:

Major architectural changes should result in a new major version.

26. FUTURE INVENTION DISCLOSURE REGISTER

Potentially novel technical developments should be separately documented.

- INVENTION ID:
- TITLE:

- INVENTOR(S):
- DATE OF CONCEPTION:
- DATE OF REDUCTION TO PRACTICE:
- DESCRIPTION:
- PROBLEM SOLVED:
- TECHNICAL ADVANTAGE:
- PRIOR ART REVIEW:
- RELATED IP ASSETS:
- SOURCE CODE:
- DRAWINGS:
- PUBLIC DISCLOSURE:
- PATENT REVIEW:
- STATUS:

Public disclosure before appropriate patent strategy may affect patent rights in some jurisdictions. Patent strategy should therefore be reviewed before public disclosure of potentially patentable inventions.

27. LEGAL AND IP LIMITATIONS

This dossier is a technical and organizational record.

It should not be interpreted as establishing that:

- every concept described is patentable;
- every name is legally available as a trademark;
- every component is automatically owned by Rounsaville Technologies;
- a conceptual architecture is automatically protected by copyright;
- third-party dependencies become proprietary;
- a dated document alone establishes exclusive ownership.

Legal ownership depends on applicable law and the facts surrounding creation, employment, contracts, assignments, licensing, and third-party materials.

For maximum protection, Rounsaville Technologies should consider obtaining qualified legal advice regarding:

- copyright registration;
- trademark searches and registration;
- patentability analysis;
- invention disclosures;
- trade-secret protection;
- contributor agreements;
- contractor agreements;
- employee IP agreements;
- software licensing;
- privacy and data protection;
- open-source compliance.

28. MASTER OWNERSHIP STATEMENT

The R-CoreX Intelligence Architecture and the systems described in this dossier are documented as part of the intellectual-property portfolio and technical architecture of:

ROUNSAVILLE TECHNOLOGIES

The architecture is attributed to:

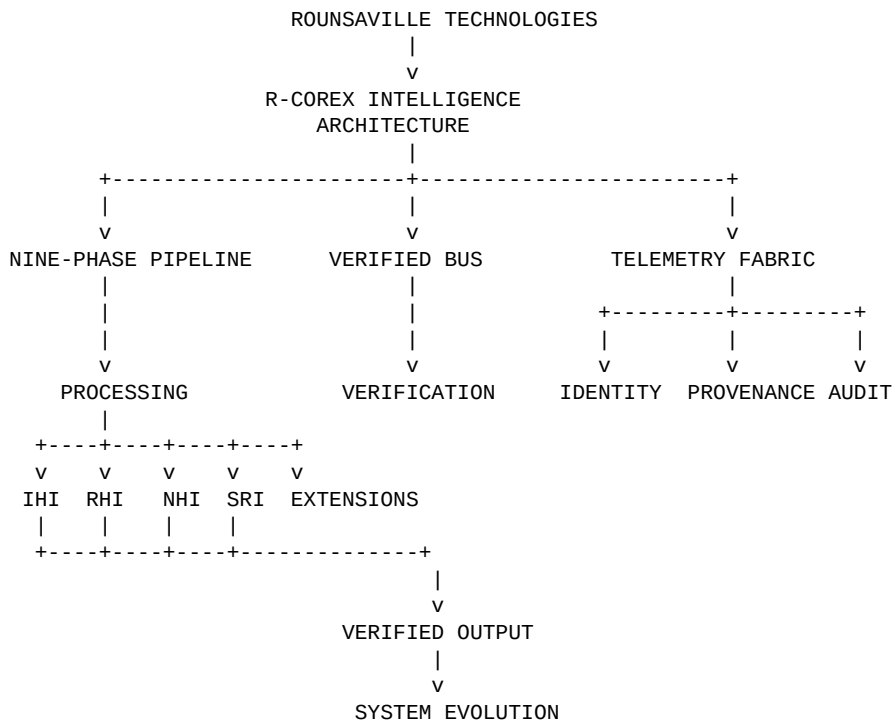
JOSEPH MICHAEL ROUNSAVILLE

as the originating author and principal architect identified in this record.

This dossier establishes the beginning of a formal provenance trail for the architecture, initially documented on **July 22, 2026 (v1.0.0)** and reconceptualized against real, git-verified repositories on **July 23, 2026 (v1.1.0)**.

Future changes, contributors, assignments, licenses, and ownership transfers should be recorded in subsequent versions and supporting legal documents.

29. MASTER ARCHITECTURE SUMMARY



30. FINAL RECORD

- **Document:** R-CoreX Intelligence Architecture — Master Intellectual Property, Technical Provenance, and Architecture Dossier
- **Organization:** Rounsaville Technologies
- **Principal Architect / Author:** Joseph Michael Rounsaville
- **Version:** 1.1.0
- **Initial Documentation Date:** July 22, 2026

- **This Revision Date:** July 23, 2026
- **Classification:** Proprietary / Confidential — Internal Master Record
- **Purpose:** Technical architecture, intellectual-property provenance, ownership documentation, version control, and future commercialization foundation.
- **Companion Record:** RT-IP-RCOREX-MAPPING-002_v1.0.0 — full code-to-concept mapping evidence.

END OF MASTER DOSSIER v1.1.0